Freight Rate Prediction

Data, validation and inference

Spotter technical assessment | 14 October 2026

Assessment objective and result

Predict a single freight load's `posted_rate` from route, distance, equipment, weight, date and supplied market signals. Deliver predictions for 12,000 unlabeled validation loads and a 31-day December price curve for a fixed Lexington–Fort Wayne load.

The selected model is CatBoost trained on ln(posted_rate), with predictions returned to rate units by exp(·). The final model uses 52 predictors and 285 trees. Both required CSV files pass the assessment scorer's structural checks.

Evaluation	CatBoost	Quote × distance
Weighted 3-fold CV MAPE	5.1119%	12.78%
October 18–31 test MAPE	5.5562%	21.028%
October test MAE	115.65	411.32
October test RMSE	706.52	838.65

Most test loads are predicted closely, while a small group of unusually priced loads dominates aggregate MAPE. No price outliers are removed. December predictions range from 813.95 to 844.65 rate units, conditional on reconstructed market and quote inputs.

Files and prediction periods

Input	Rows	Target	Period / purpose
`train-test.csv`	48,000	Known	Jan 1–Oct 31, 2025
`validation.csv`	12,000	Hidden	Nov 1–Dec 31, 2025
`december-chart-inputs.csv`	31	Predict	Fixed load, Dec 1–31, 2025

Report sequence

Inspect data quality and distribution changes; define geographic and market features; validate chronologically; optimize and explain the model; examine errors; reconstruct December inputs; and generate final files. Fold SHAP plots and validation inference diagnostics appear in the appendix.

Interpretation boundaries

Validation targets are hidden, so their accuracy cannot be measured locally. The internal test was reserved retrospectively and is not a pristine external benchmark. The market curve uses known future input values without price targets. December inputs are estimates, and the uncertainty band excludes reconstruction uncertainty.

1 Data quality before modeling

Train and validation have no duplicate load IDs or duplicate complete rows. Distances, quote signals and training targets are positive and finite. Weight and market index require cleaning. Empty prediction-template columns are expected outputs, not missing predictors.

Issue	Train (48,000)	Validation (12,000)
Missing weight	300 (0.625%)	165 (1.375%)
Nonpositive weight	292 (0.608%)	145 (1.208%)
Missing market index	374 (0.779%)	249 (2.075%)
Duplicate load IDs	0	0

Nonpositive weights become missing. Weight and market medians fit only the corresponding training partition, including within CV folds. Cleaning preserves row order, IDs and supplied coordinate fields. No target outliers are deleted.

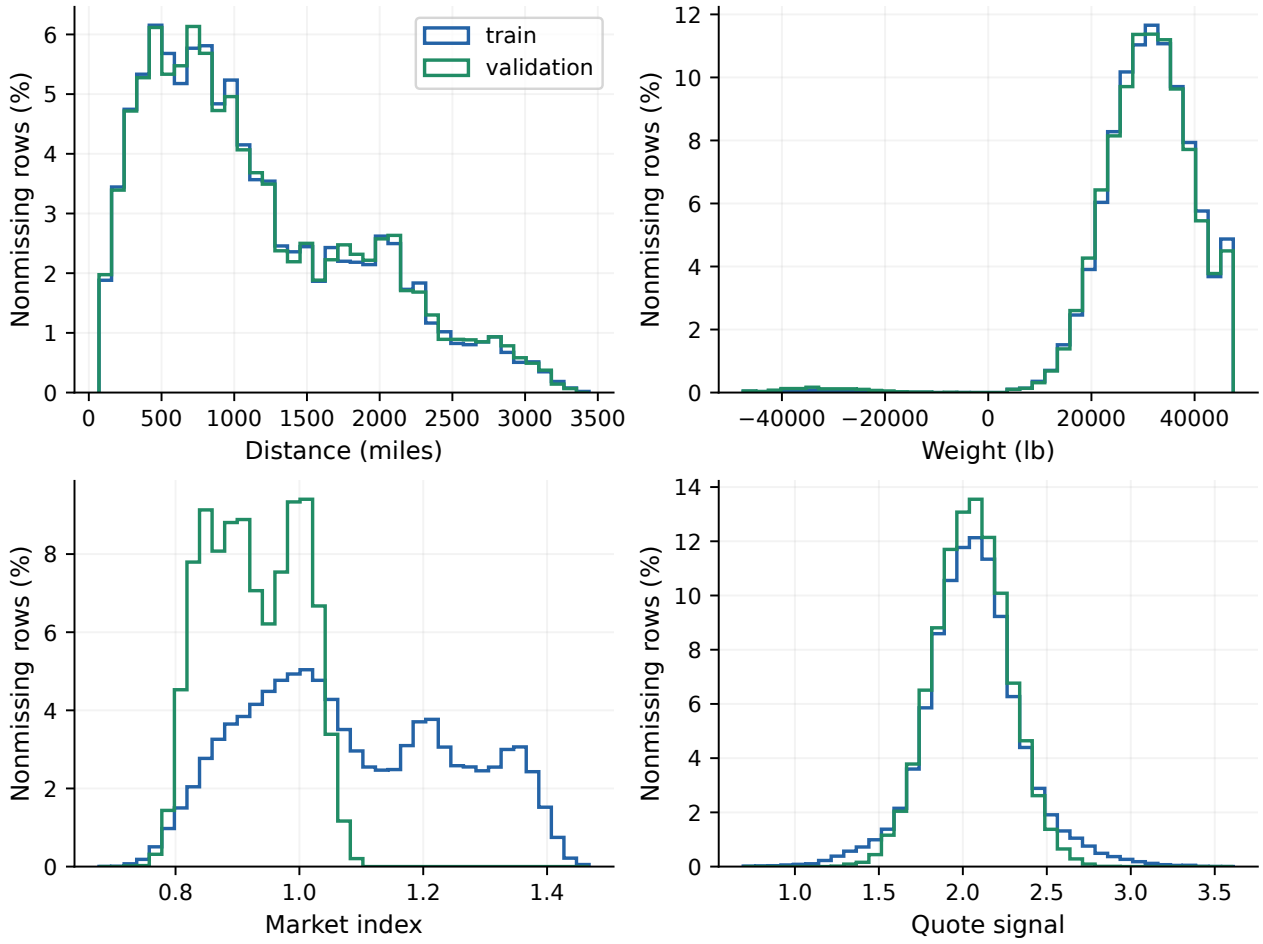


Figure 1: Selected raw numerical distributions, independently normalized by dataset. Negative weights are visible before cleaning. Missing values are excluded from histograms, not hidden in the audit.

The training target spans 57.22–25,533.00, with median 2,030.76; its long tail motivates log-target fitting. Distance medians barely change: 953.3 versus 953.6 miles. Quote medians are 2.056 versus 2.051. Mean market index changes from 1.083 in train to 0.927 in validation, motivating explicit analysis of the market process rather than assuming identical distributions.

2 Temporal signals and sampling shift

The panels below compare market index, quote signal and observed price globally and for trips from/to Detroit. Validation is restricted to known endpoint cities; its actual prices are unavailable.

Train spans 304 days and validation 61. Mean sample count rises from 157.89 to 196.72 loads per day. Restricting validation to known pickup/delivery cities leaves 10,553 rows, or 173.00 per day. Novel cities explain part, but not all, of the sample-count gap.

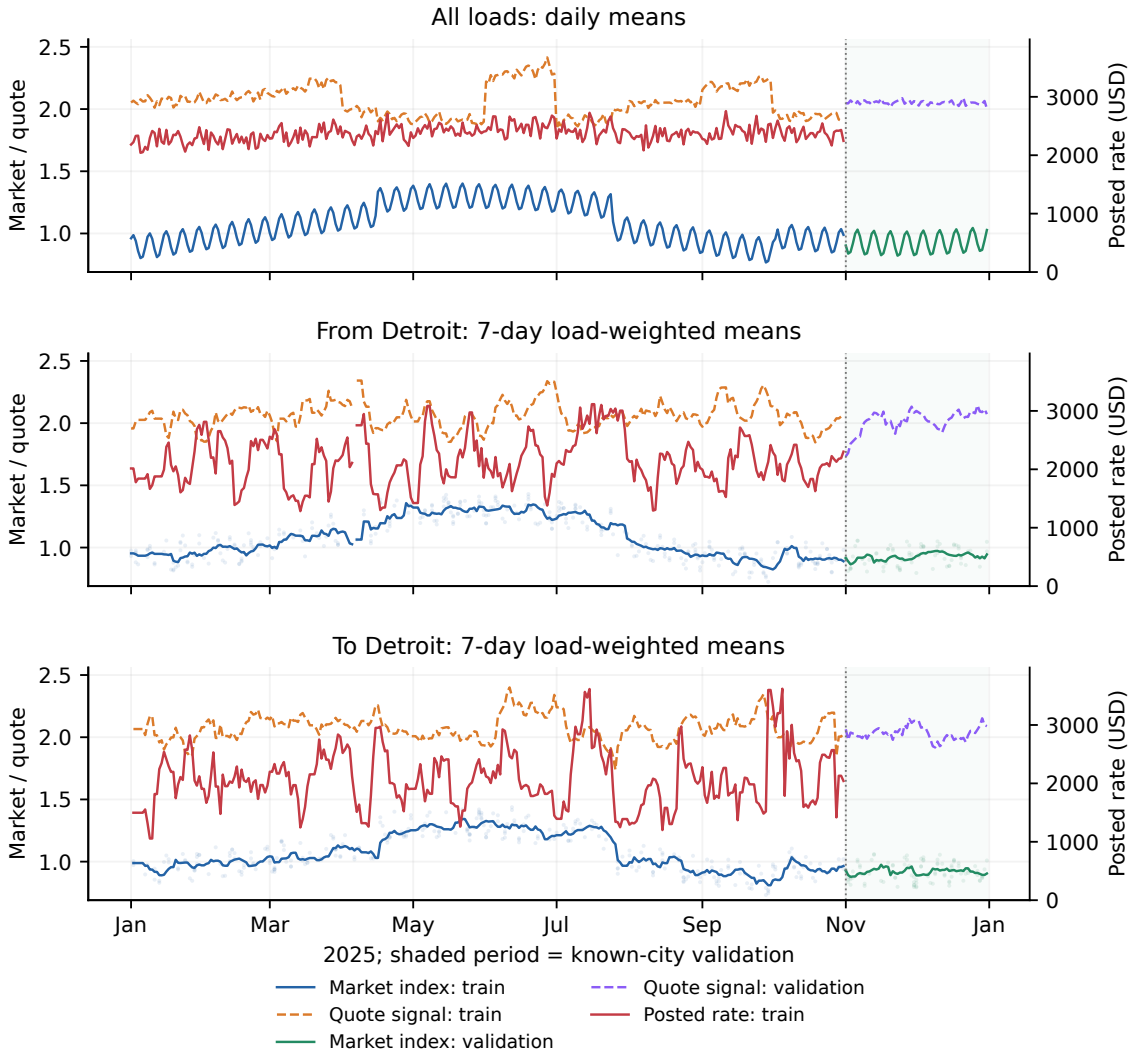


Figure 2: The three signals from EDA: global daily means, then trailing 7-calendar-day load-weighted means from and to Detroit, separately within each split. Market and quote use the left axis; posted rate uses the right. Quotes are dashed; faint Detroit points are individual market observations. Validation has no posted-rate curve.

Each endpoint role has 64 training cities and 72 validation cities. The eight names absent from the union of training endpoints are Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk and San Diego. The known-city filter checks each endpoint's training role separately.

These are sampled records, not verified counts of the entire market. Higher sampling density alone does not prove an increase in real freight demand. City features therefore include normalized shares and directional imbalance. The temporal panels expose market regimes and differences in the signals without establishing causation. The analysis does not show that new cities cause the market-index shift.

City composition and new validation locations

The map summarizes the same known-city validation subset used in the temporal comparison. Positions use the separate GeoNames city reference, rather than the displaced coordinates supplied with the freight records.

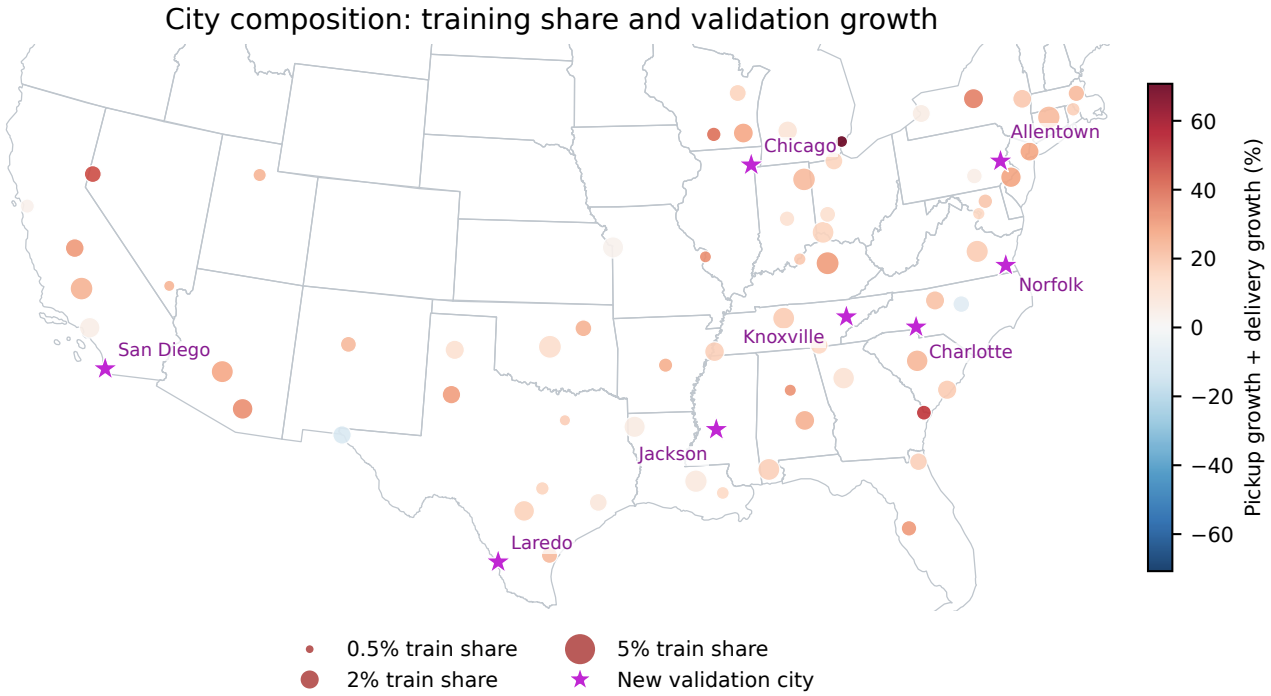


Figure 3: Circle area represents a city's share of all training pickup and delivery endpoints. Color shows the sum of pickup and delivery percentage changes in mean daily loads: red indicates growth, blue decline. Magenta stars identify the eight cities absent from both training endpoint roles.

For each endpoint role, growth is (validation loads/day/training loads/day − 1) × 100%. The color adds these two percentages; it is not the percentage change of their combined count. Counts use the full calendar length of each split, including days without loads for a city.

New cities have zero training share and undefined percentage growth, so they receive fixed-size stars. Their presence is determined from the full validation set; the colored circles use validation restricted to known pickup and delivery cities. Marker areas encode training composition, not city population or geographic size.

3 Geographic audit and routing choices

The dataset supplies one coordinate pair per named city, constant across endpoint roles and datasets. It does not reveal individual warehouse locations within a city. Against matched official city boundaries, 70 of 72 points lie outside their named municipality; 52 are more than 100 km outside.

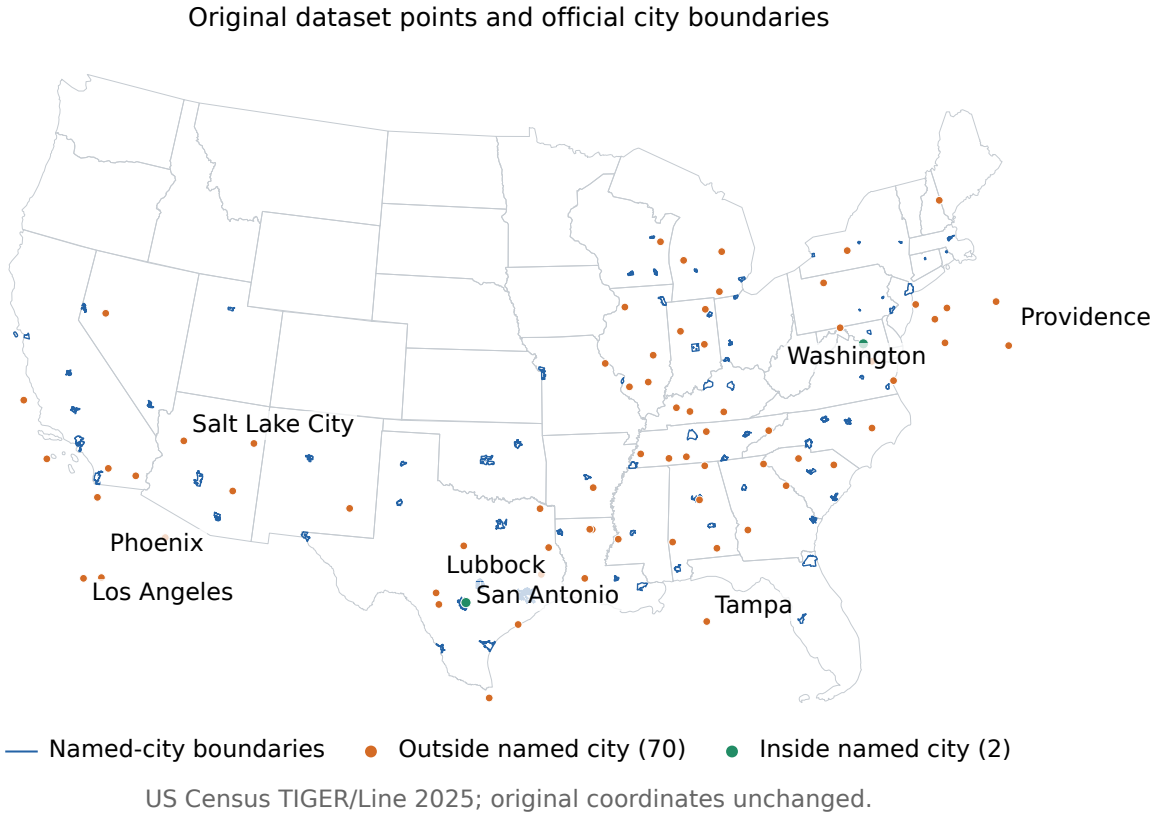


Figure 4: Original supplied points and US Census named-city boundaries. Outside the named city does not mean outside every municipality, nor prove that a coordinate is incorrect.

Original coordinates remain model inputs. Road features use a separate GeoNames city reference with a documented Washington override: routing displaced supplied points would answer a different geographic question. References are not claimed to be real warehouse sites. References and directed route tables are saved with preprocessing. Inference needs no live routing API. Ambiguous city names use selected reference records; geographic matching remains an assumption. External data provenance is listed in the final appendix.

4 Feature engineering

The model uses 52 features, including three categorical columns: pickup, delivery and equipment. Load IDs, raw date metadata, targets and output fields are excluded.

Group	Count	Information
Original predictors	11	Cities, equipment, supplied coordinates, distance, weight, market and quote.
Cyclical time	7	Sine/cosine of weekday, within-month and within-year phase; weekend flag.
Road routes	5	HERE miles/hours, road-to-air and input-to-road ratios, availability.
Market decomposition	16	Trend, harmonic, expectation, derivatives, residual statistics and five regime coefficients.
US holidays	7	Actual/observed flags, distance to adjacent holidays and nearby-day indicators.
Historical city inputs	6	Endpoint shares, directional balance and destination/origin diversity.

City aggregates use previous days only for training; the entire current day is excluded before updates. Test and validation use frozen training history. No target encoding or price-derived aggregate enters the predictors. Calendar holidays are known in advance; there is no raw elapsed-date feature.



Figure 5: Cached HERE truck geometries for Dallas–Houston, Detroit–Atlanta and Los Angeles–New York. Routing estimates are contemporary static references, not verified historical 2025 journeys.

Route distances and durations augment the supplied load inputs without replacing their original coordinate columns. Correlated distance features are interpreted jointly in the SHAP analysis later in the report.

5 Market-index decomposition

The observed daily market mean is fitted by up to four chronological regimes:

$$f_k(t) = a_k + b_k t + c_k t^2 + u_k \sin(\theta) + v_k \cos(\theta), \qquad t = \frac{\text{days since 2025-01-01}}{365.25}, \quad \theta = \frac{2\pi\,\text{weekday}}{7}.$$

Breakpoints minimize daily squared error; BIC selects the number of segments. A minimum of seven observed days permits a new short regime without forcing it to absorb the previous one. Global daily-mean fit RMSE is 0.002588 index units.

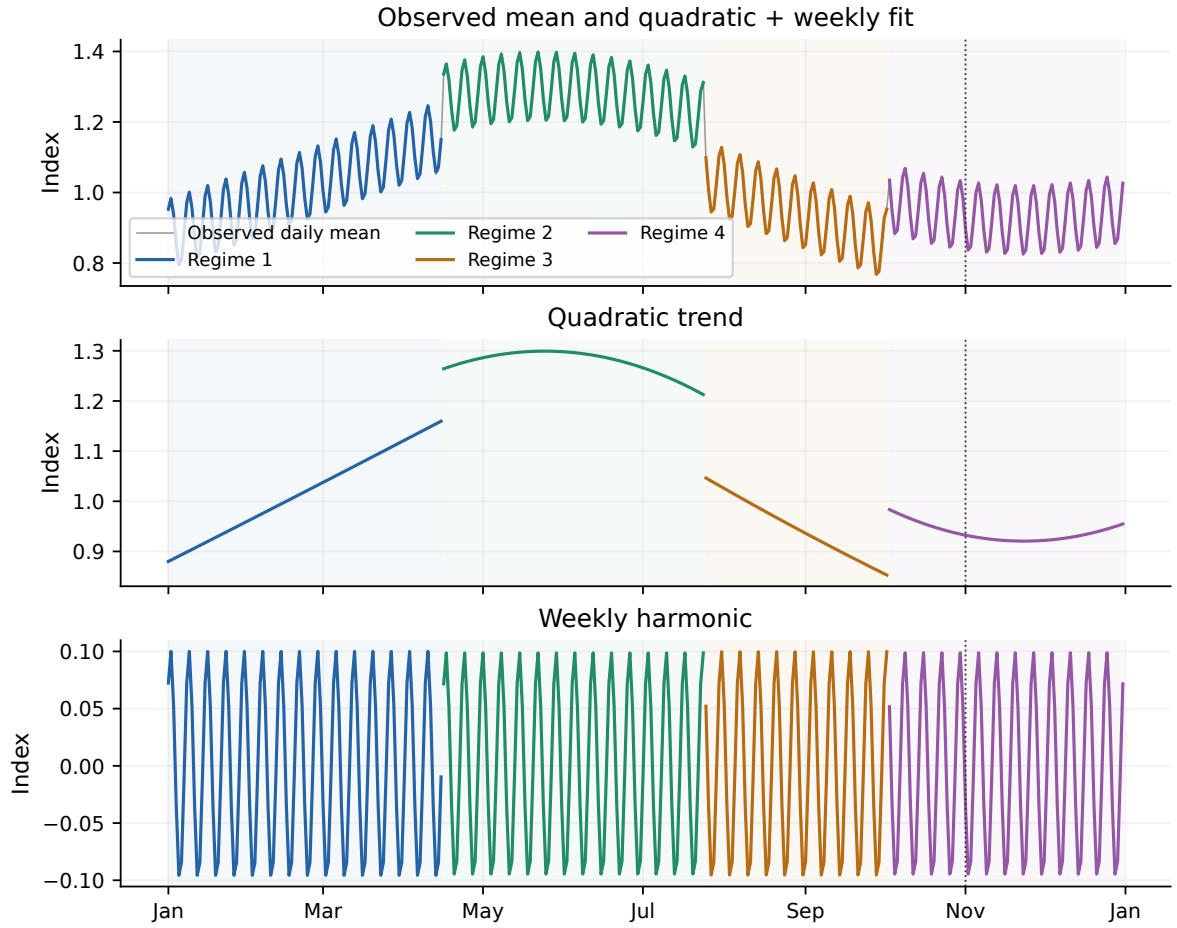


Figure 6: Fitted expectation, quadratic trend and weekly harmonic. The dotted date marks supplied validation. Its known market inputs participate in this descriptive fit.

Regime dates	a	b	c	u	v
Jan 1–Apr 15	0.88003	0.96751	0.05180	0.05497	-0.08454
Apr 16–Jul 24	0.81194	2.48109	-3.15744	0.05449	-0.08337
Jul 25–Oct 2	1.79519	-1.56563	0.41259	0.05485	-0.08427
Oct 3–Dec 31	3.47174	-5.71274	3.19806	0.05439	-0.08352

The equation uses known train plus validation *inputs*, without price targets. Residual reference distributions and equipment/lane residual statistics fit each training partition only. This covariate-informed batch setup is not a strict simulation of online forecasting without future market inputs; deployment would require a separate input-availability policy.

6 Chronological validation protocol

The final 14 labeled days, October 18–31, form an internal test of 2,183 loads. Development contains 45,817 earlier rows. Expanding folds split unique dates, keeping a day's loads together.

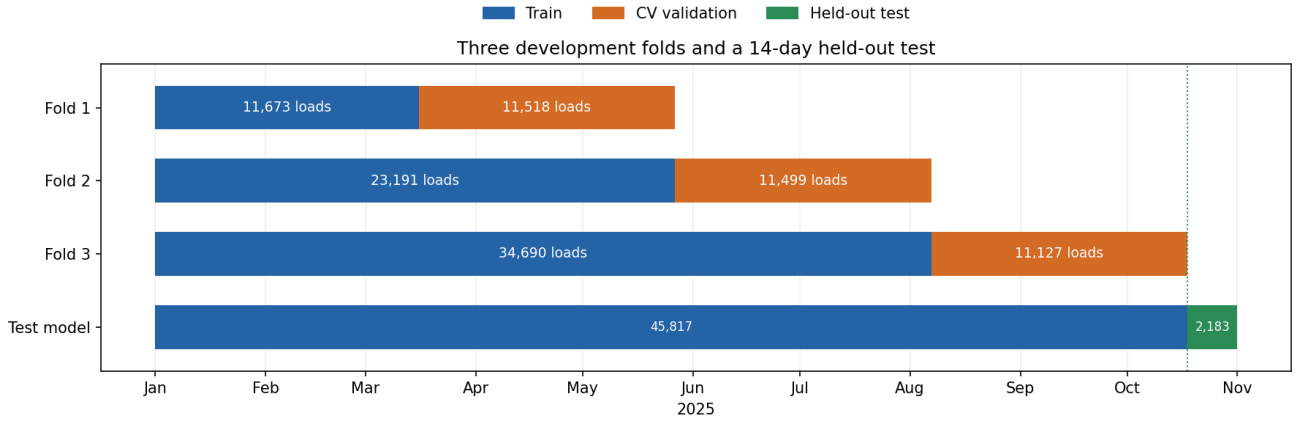


Figure 7: Three development folds and the separate holdout. There is no random mixing of days across a train/validation boundary.

Fold	Training dates	Validation dates	Train	Valid
1	Jan 1–Mar 15	Mar 16–May 26	11,673	11,518
2	Jan 1–May 26	May 27–Aug 6	23,191	11,499
3	Jan 1–Aug 6	Aug 7–Oct 17	34,690	11,127

Imputation, market-residual distributions and city histories fit each fold's training rows. The shared market equation is the disclosed exception: it uses known future covariates, never price targets.

Optuna minimizes original-scale MAPE with recency weights:

$$J = 0.2\,\mathrm{MAPE}_1 + 0.3\,\mathrm{MAPE}_2 + 0.5\,\mathrm{MAPE}_3.$$

Here $\mathrm{MAPE} = \frac{100}{n}\sum_i |y_i - \hat{y}_i|/y_i$; a value of 1% means mean relative error, not classification accuracy. No pooled OOF predictions or score are computed. These windows also choose parameters and early stopping, so CV scores are development estimates after selection.

The test model fits development only, using the CV-selected fixed tree count; its labels are read for evaluation afterward. A separate submission model then refits all 48,000 labeled rows. That full-train model predicts supplied validation, not the internal test.

The holdout dates appeared in earlier experiments and were reserved retrospectively. Its score is informative, but not equivalent to a pristine one-time external evaluation. Fixed boundaries and this disclosure are retained throughout the report.

7 Optimization and selected parameters

The active Optuna study has 50 completed TPE trials, seed 42. Each trial trains on $\ln(y)$ with RMSE loss and log-RMSE early stopping after 100 rounds, up to 1,500 trees. Selection uses weighted original-scale MAPE.

Search covers depth 4–8; learning rate 0.02–0.15; L2 0.1–30; random strength 0.001–3; column sampling 0.4–1; borders 64/128/254; SymmetricTree, Depthwise and Lossguide; Newton/Gradient leaf estimation; 1–10 leaf iterations; and No/AnyImprovement backtracking. Leaf-size and leaf-count parameters are conditional on growth policy.

Parameter	Value	Parameter	Value
Depth	6	Learning rate	0.0569703
L2 leaf regularization	2.28909	Random strength	1.16862
Column sampling	0.977059	Global borders	254
Growth policy	SymmetricTree	Leaf method	Newton
Leaf iterations	1	Final trees	285

Backtracking is AnyImprovement. Fold stopping counts are 218, 285 and 427; their median, 285, fixes test and submission tree counts. Active-study trial 41 is selected. SQLite study names fingerprint prepared fold values and configuration.

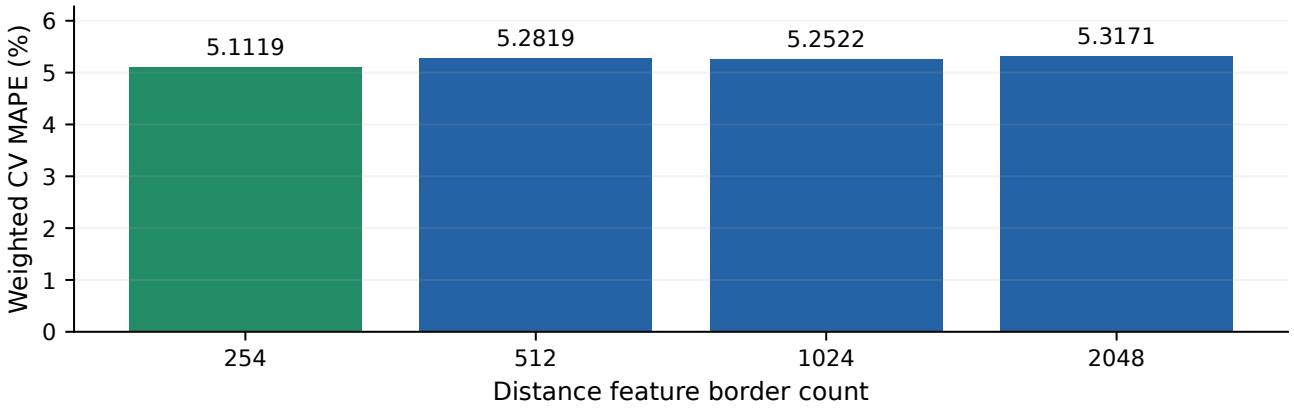


Figure 8: Extra quantization search for golden feature `distance`, with other settings fixed. The current pipeline selects 254; additional bins do not improve it.

`golden_features` accepts numeric column names and selects a separate border count for each, sequentially under the same objective. Only results from the current feature/data fingerprint are reported; earlier pipeline experiments are not mixed into this comparison.

8 Performance against the product baseline

The direct baseline is $\hat{y} = \text{quote_signal} \times \text{distance}$, without a fitted multiplier or correction. Both methods are evaluated on identical rows.

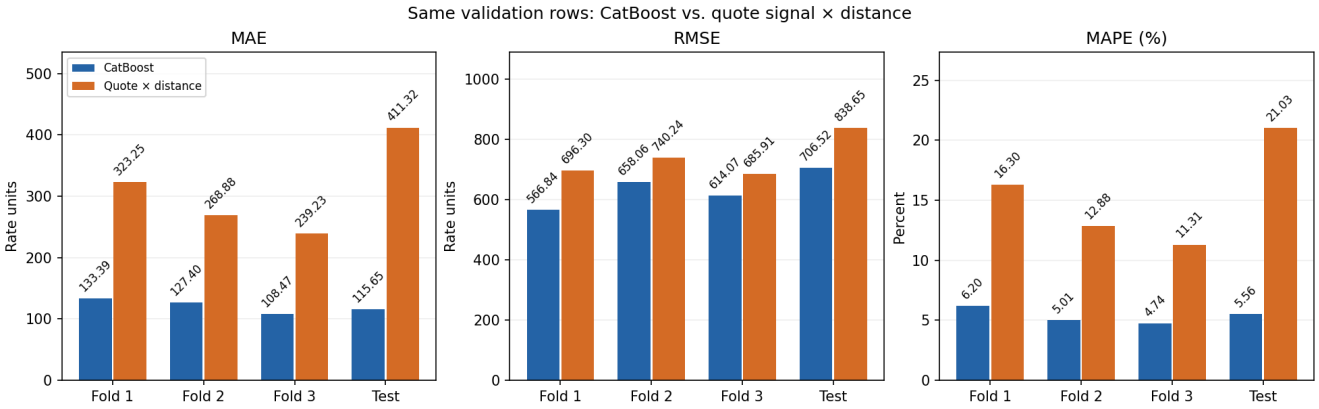


Figure 9: CatBoost and the product baseline: MAE, RMSE and MAPE improve on every split.

Split	CatBoost MAPE	Product MAPE	CatBoost MAE	Product MAE
Fold 1	6.200%	16.298%	133.387	323.250
Fold 2	5.006%	12.881%	127.400	268.881
Fold 3	4.740%	11.310%	108.474	239.234
October test	5.556%	21.028%	115.648	411.320

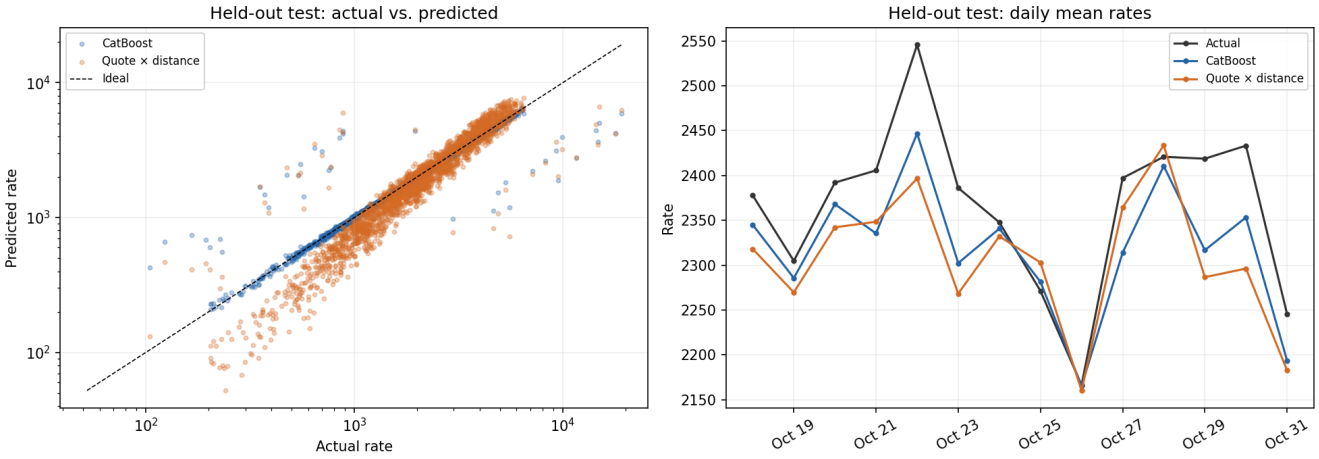


Figure 10: Individual predictions and daily means on the test. Similar daily mean rates do not imply equal load-level accuracy: errors can cancel during averaging.

Test median APE is 1.897%, P90 APE is 4.624%, and 97.939% of loads are within 10% of observed price. RMSE remains sensitive to large deviations. The next pages examine these cases without excluding them from headline metrics.

9 Residual diagnostics

Residual is $y - \hat{y}$: positive means underprediction. All test diagnostics use the development-only model, not the full-train submission model.

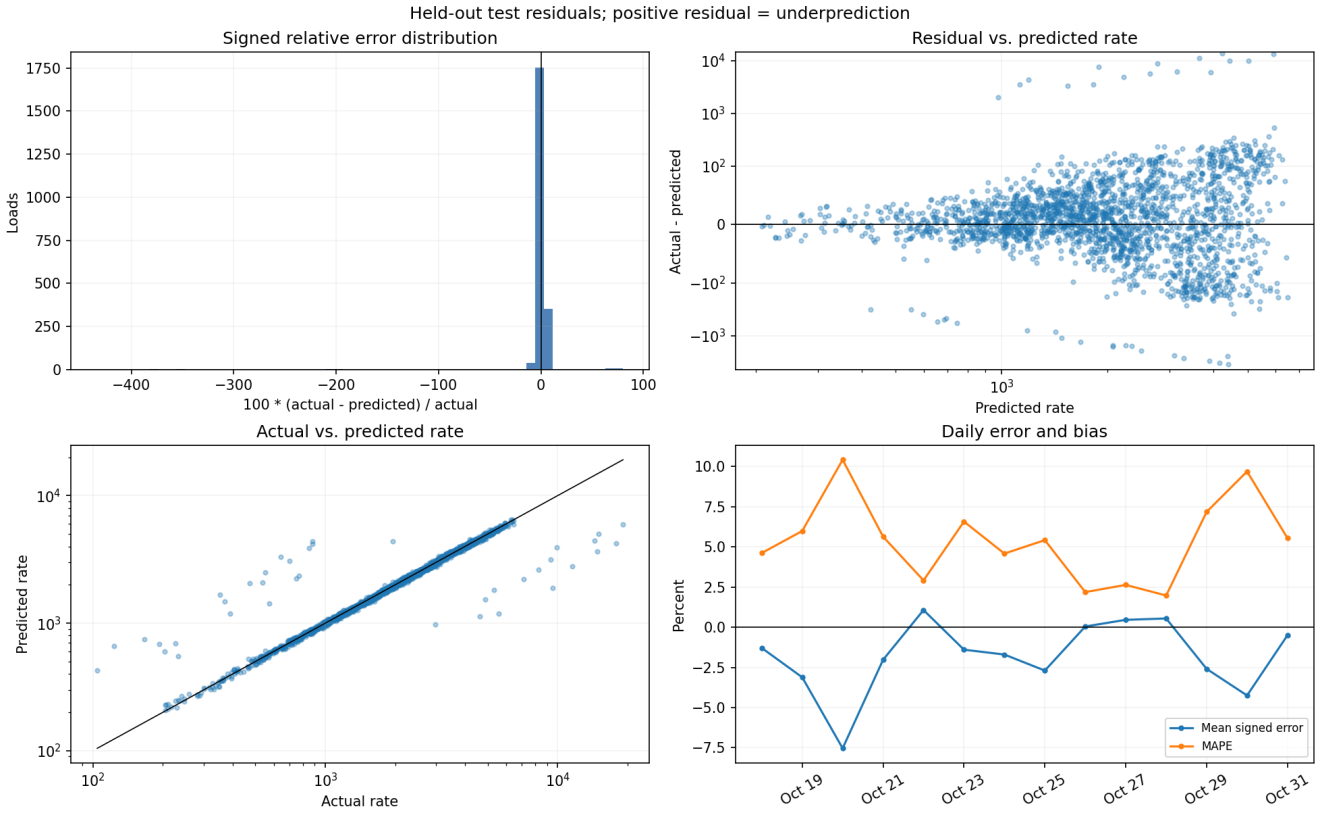


Figure 11: Signed percentage errors, residual magnitude, actual versus predicted price and daily errors. Log and symmetric-log axes expose extremes without deleting rows.

Most predictions track actual rates closely, but extreme errors occur in both directions. Price-scale errors tend to grow with price; relative metrics therefore complement MAE and RMSE. Daily spikes may reflect a few loads rather than a uniform bias.

Test group	Rows	Mean APE	Total APE share
APE at most 10%	2,138	2.163%	38.131%
APE 10–50%	7	10.945%	0.632%
Severe overprediction	22	286.162%	51.904%
Severe underprediction	16	70.756%	9.334%

The 38 loads above 50% APE are 1.74% of rows but contribute 61.24% of summed percentage errors. The 2.163% ordinary-case metric is diagnostic, not an alternative overall score. Removing rows based on their test errors would bias evaluation; overall MAPE stays 5.556%.

10 What distinguishes the largest errors?

All severe-error lanes were observed in development. Median historical lane frequency is about 14, comparable with ordinary cases. Weight, quote and route ratios overlap the rest of the data; unseen geography does not explain these particular errors.

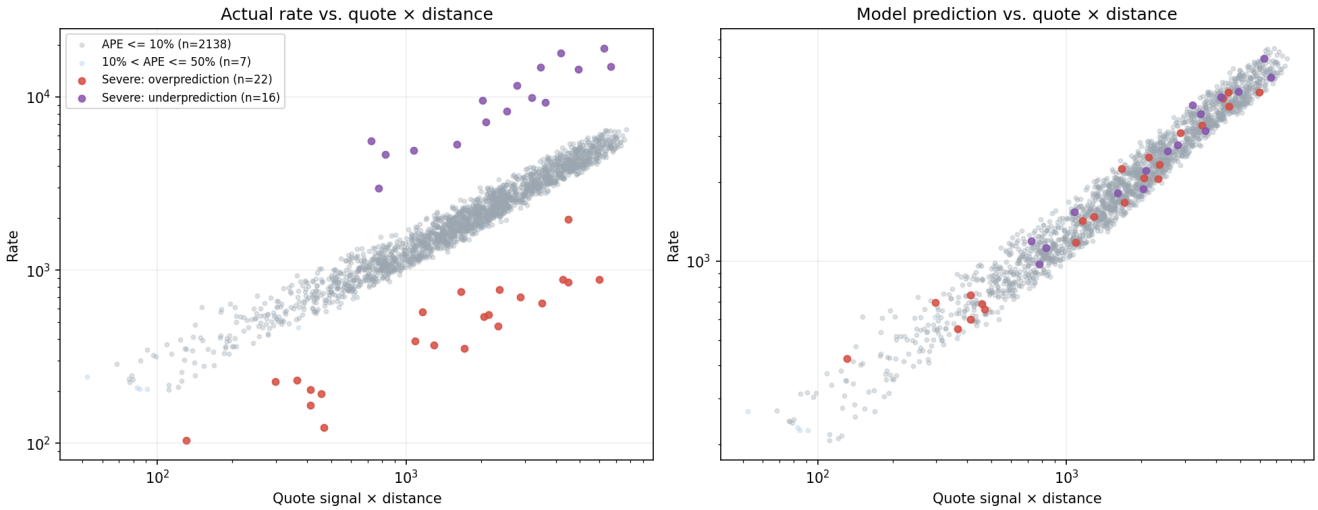


Figure 12: Severe-error targets separate above and below the ordinary price relationship, while predictions stay closer to it. Target-derived ratios are diagnostics only.

For each of the five worst cases, 20 approximate historical peers share equipment and are nearest in standardized distance, weight, quote and market inputs. Matching never uses targets or prediction errors; peers are not necessarily the same lane.

Load ID	Actual	Predicted	Peer median	APE
TR-046400	123.32	657.79	852.36	433.40%
TR-046179	643.58	3,292.46	2,970.97	411.59%
TR-046174	880.13	4,404.34	4,282.88	400.42%
TR-046260	352.82	1,677.18	1,617.79	375.36%
TR-047504	880.04	4,167.31	4,185.07	373.54%

These actual prices are 14–22% of approximate peer medians. Median actual/predicted is 0.248 for severe overprediction and 3.225 for severe underprediction. Their median quote signals, roughly 1.94–2.02, resemble ordinary cases at 1.99.

An unobserved pricing factor or target anomaly is plausible, not established. Small city/equipment enrichments do not prove a reliable input-based detection rule. The analysis neither establishes an irreducible error floor nor proves that 1% MAPE is mathematically impossible.

11 SHAP interpretation of the final model

Native CatBoost contributions explain raw model output:

$$\ln(\hat{y}_i) = \phi_{0,i} + \sum_j \phi_{ij}.$$

Positive values raise predicted log rate; negative values lower it. Explanations pass numerical additivity checks. Feature ranking uses all explained rows; points show a deterministic sample of up to 1,500 loads and the top 20 features.

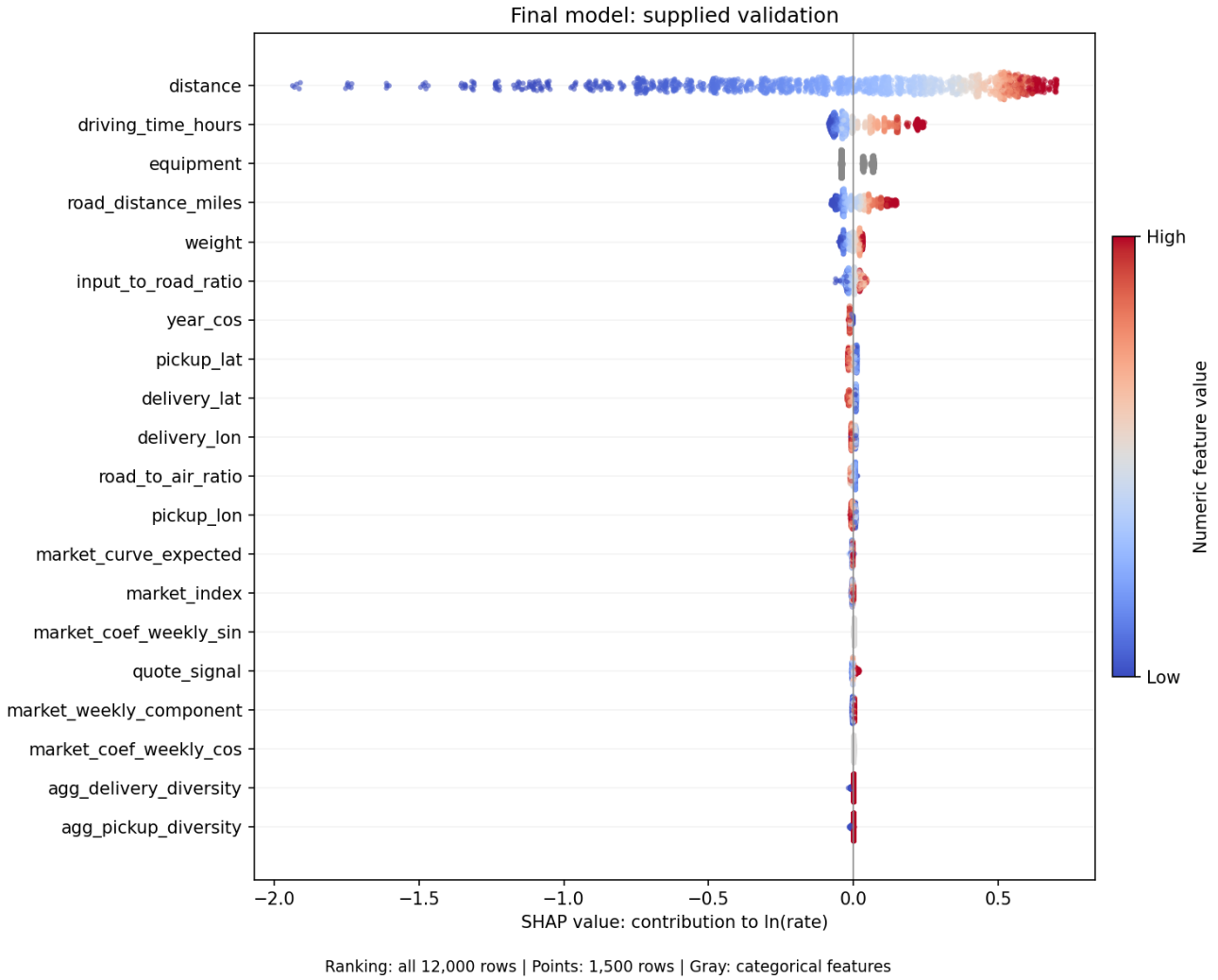


Figure 13: Full-train model explanations on supplied validation. Numeric colors indicate low-to-high values; categorical points are gray. No validation targets are needed.

Distance and route-duration effects agree with their predictive role. Correlated features should be read jointly. A small quote contribution does not prove that quote is useless or absent from interactions. Fold explanations follow in the appendix.

12 Decision paths for the five worst test loads

Paths explain the development-only model, accumulating contributions from expected log rate to each prediction. Features outside the top 20 are summed into one row, preserving additivity.

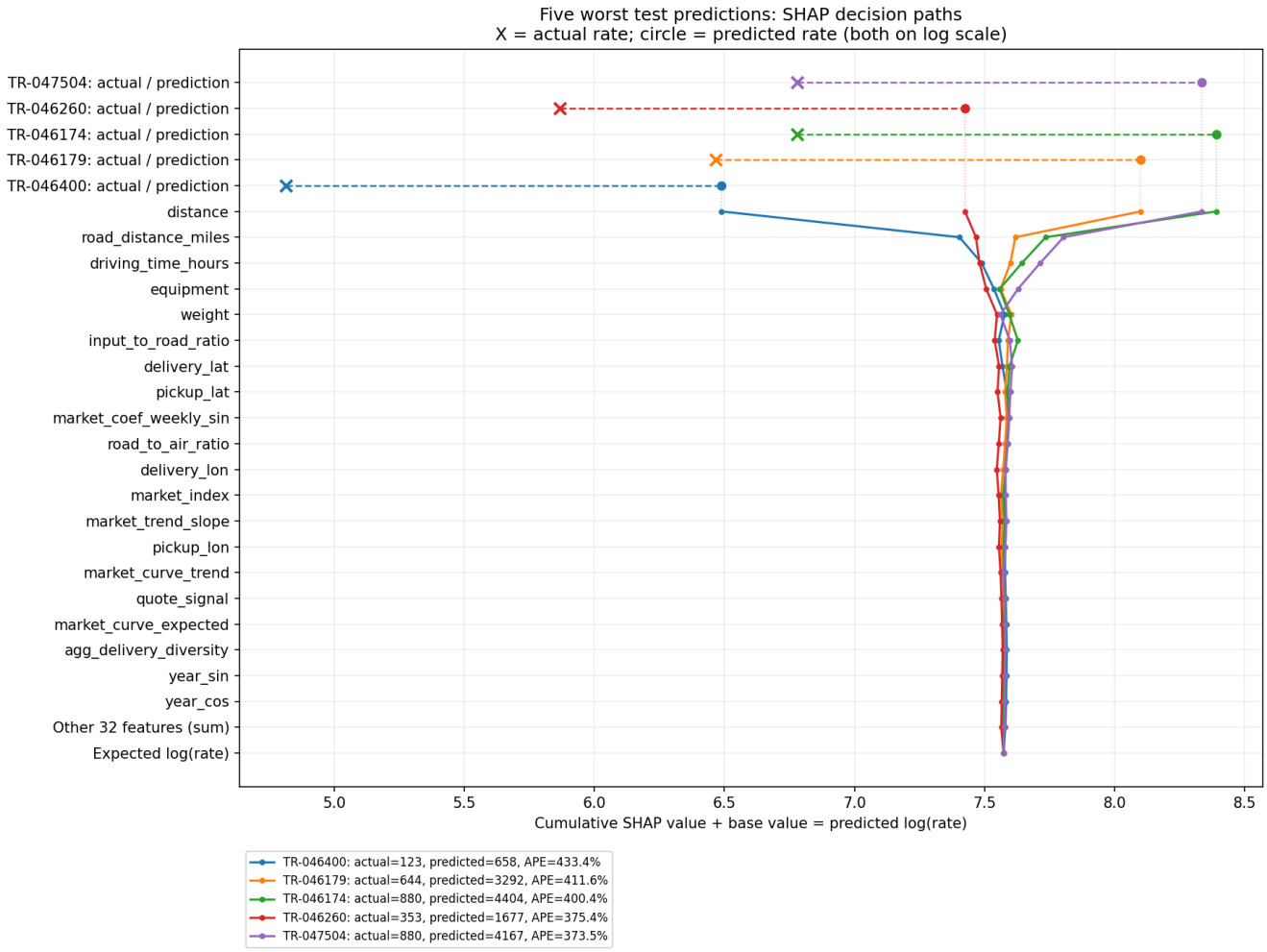


Figure 14: Top circles mark predicted log prices and crosses mark actual log prices. Horizontal gaps are errors, not SHAP contributions. Targets are shown only for comparison.

The model reacts to route length, but these actual prices are far lower than predicted. SHAP explains model behavior; it does not establish why the observed price differs or prove that a target is wrong.

13 Reconstructing the fixed December inputs

The prescribed load is Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Dates are supplied; coordinates, quote and market are absent. Unique source city coordinates are Lexington (36.99152, −84.99876) and Fort Wayne (41.31561, −85.36206). Routing references remain separate.

Validation has 12 loads on this directed lane, eight in December. Three December loads also use Dry Van. Their signal values are copied on those dates. They have different weights/distances from the prescribed load, so the transferred signals are proxies for the fixed-load scenario.

Date	Market index	Quote signal	Source load ID
Dec 5	0.97834	1.92127	TE-006705
Dec 9	0.92830	2.10398	TE-007537
Dec 21	0.81167	2.33895	TE-009851

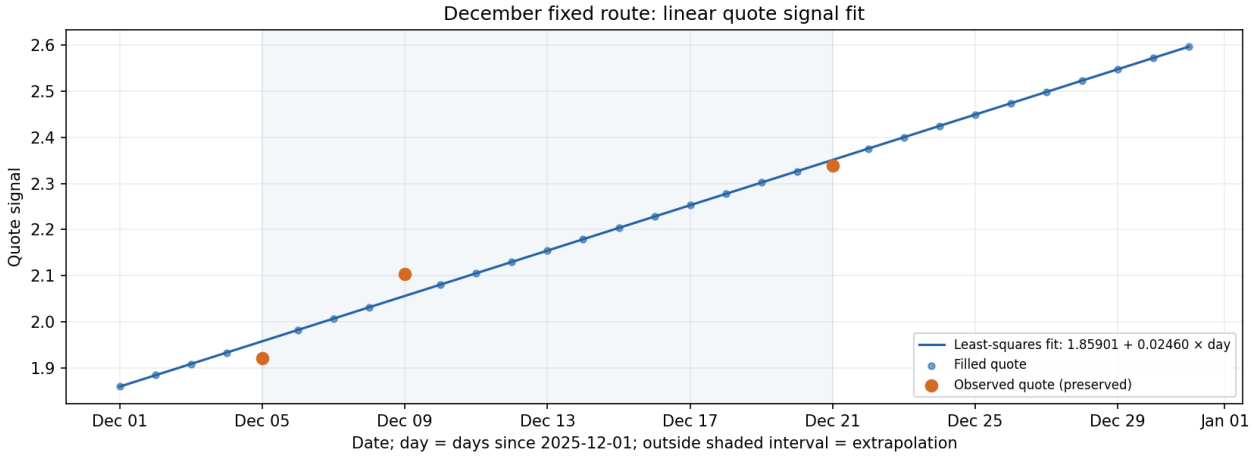


Figure 15: Missing quotes use $q(d) = 1.859006 + 0.0245994d$, with d days since December 1. Observations stay unchanged; values outside their date span are extrapolated.

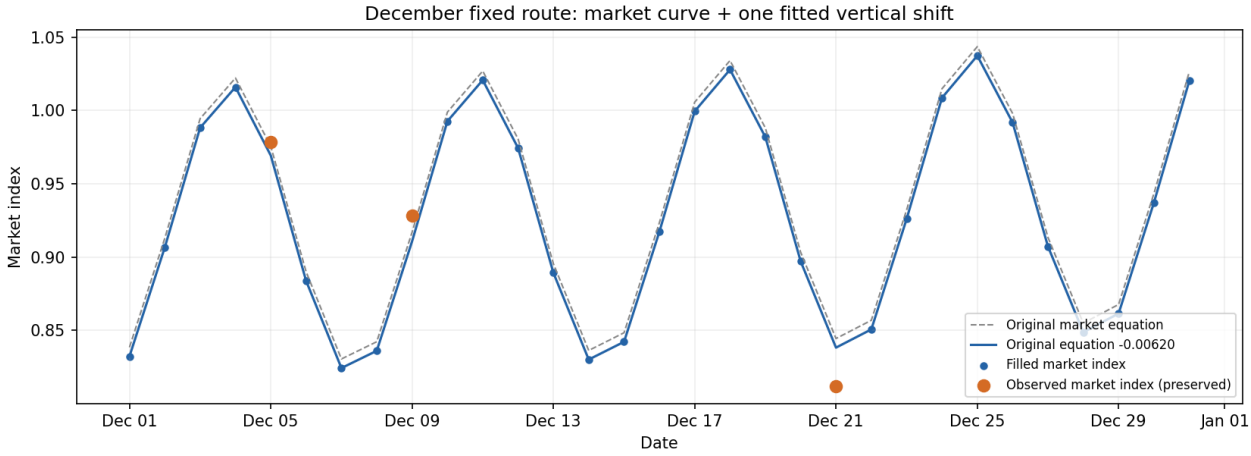


Figure 16: Missing market values use the existing equation plus $\delta = -0.006200$, the mean observed residual. Three-point fit RMSE is 0.01910; original values are preserved.

No price target enters reconstruction. Only three matched observations are available, so this is an uncertain input scenario, not a separately validated lane-level forecast.

14 December predictions and uncertainty

All 52 model features are constructed from completed inputs and saved state. Inference does not refit the model or request routes. Point prices span 813.95–844.65, with mean 825.12.

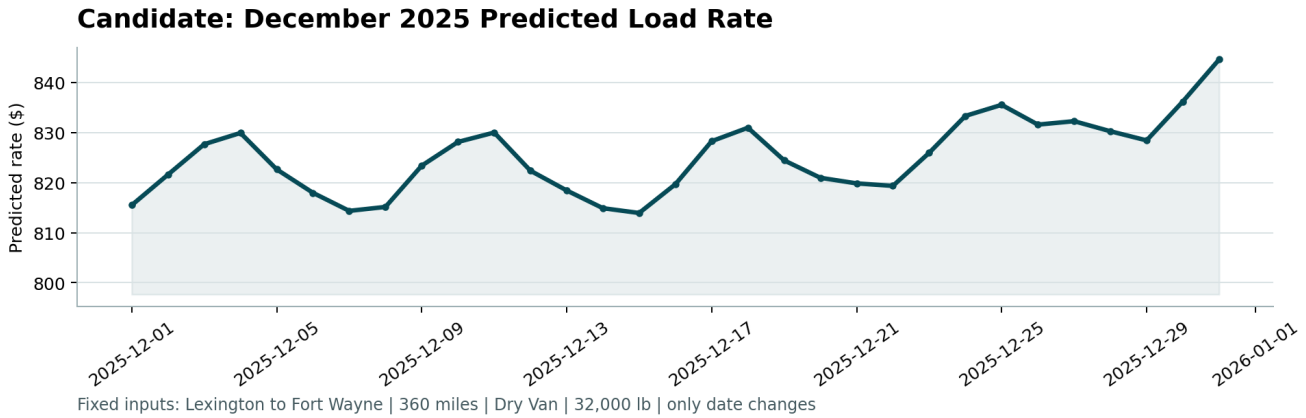


Figure 17: The required chart, generated by the assessment scorer from the completed seven-column December file. Prescribed raw load attributes are fixed; only date varies.

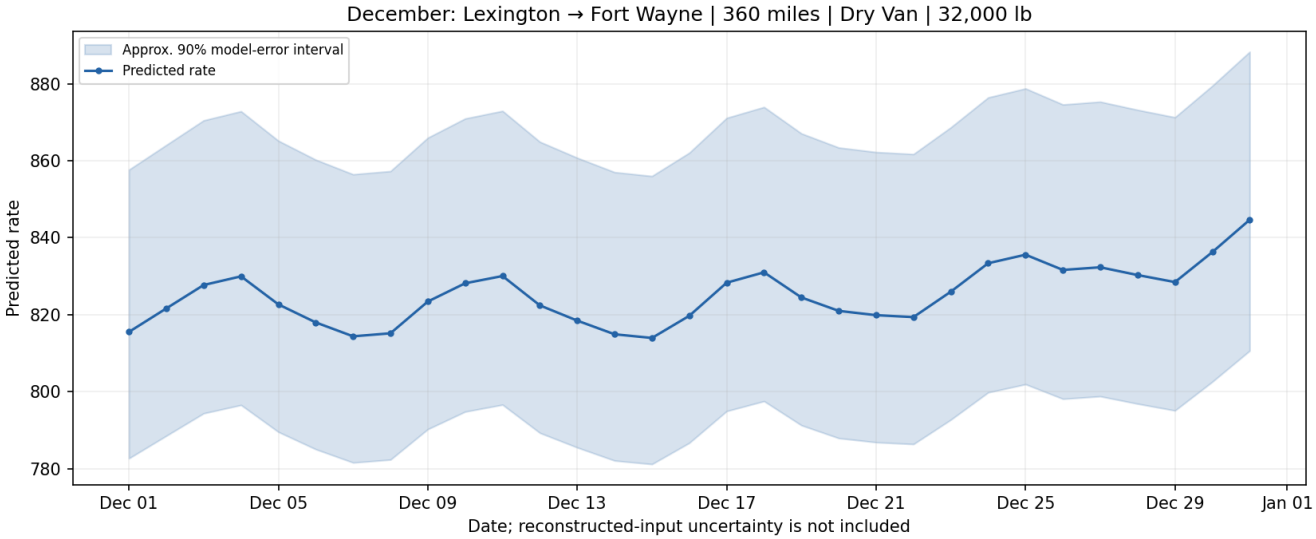


Figure 18: Approximate central 90% model-error band transferred from the October test. This is not a guaranteed conditional prediction interval.

The 5th/95th empirical quantiles of actual/predicted on 2,183 holdout loads give

$$[\hat{y}Q_{0.05}, \hat{y}Q_{0.95}] = [0.959735\hat{y}, 1.051664\hat{y}].$$

Coverage on that calibration sample is 89.922%; this is not an independent coverage test. Transfer across models, periods and lanes is an assumption.

The band excludes uncertainty in reconstructed market/quote values, the base market curve and geographic references. Input-uncertainty Monte Carlo propagation was discussed but not implemented. This band must not be labeled total uncertainty or a calibrated 90% guarantee for the fixed lane.

15 Reproducibility, outputs and limitations

1. `notebooks/eda.ipynb`: quality, distributions, temporal comparisons and geographic audit.
2. `notebooks/modeling.ipynb`: features, folds, tuning, metrics, SHAP, residuals and artifact saving.
3. `notebooks/inference.ipynb`: artifact loading, validation predictions, December reconstruction and predictions, scorer.

Shared helpers are in `notebooks/utils.py`; reusable `TimeSeriesCV` and `HParamsOpt` are in `notebooks/templates.py`. `scripts/build_report.py` rebuilds report figures and evidence without training.

Artifacts under `outputs/`

Artifact	Purpose
`final-model/model.cbm`	Native full-train model.
`final-model/preprocessing.pkl`	Feature state/order, categories and routing references.
`final-model/metadata.json`	Parameters, versions, dates, metrics and interval calibration.
`validation-predictions.csv`	12,000 unique IDs and positive finite rates.
`december-predictions.csv`	31 fixed-route rows in the original seven-column schema.
`december-prediction-intervals.csv`	Separate lower and upper estimates.
`scorer_results/candidate_december.png`	Required December chart.

Run from the repository root

```
python -m pip install -r requirements.txt
# Execute modeling.ipynb, then inference.ipynb in order.
python notebooks/score.py \
  --predictions outputs/validation-predictions.csv \
  --december-predictions outputs/december-predictions.csv \
  --output-dir outputs/scorer_results
python -m unittest discover -s tests
python scripts/build_report.py
cd output/pdf
pdflatex -interaction=nonstopmode freight_rate_report.tex
```

Use a notebook-capable environment; the verified runtime includes Jupyter/nbclient. Native-model reload checks preserve predictions. Saved versions: CatBoost 1.2.10, NumPy 2.1.3, pandas 2.3.3, scikit-learn 1.8.0. Evidence JSON includes source-artifact SHA-256 hashes.

The scorer checks file contracts, not hidden validation accuracy. CV reuses selection windows; the holdout is retrospective; future market covariates are used; December inputs are reconstructed. These limits accompany the scores. GitHub publication and a 2–3 minute Loom walkthrough remain separate submission steps.

A Fold 1: held-out SHAP values

Training: January 1–March 15; validation: March 16–May 26. Imputation and residual statistics fit this fold's training rows; the market equation uses known future covariates. Fold MAPE is 6.200%, with 218 trees.

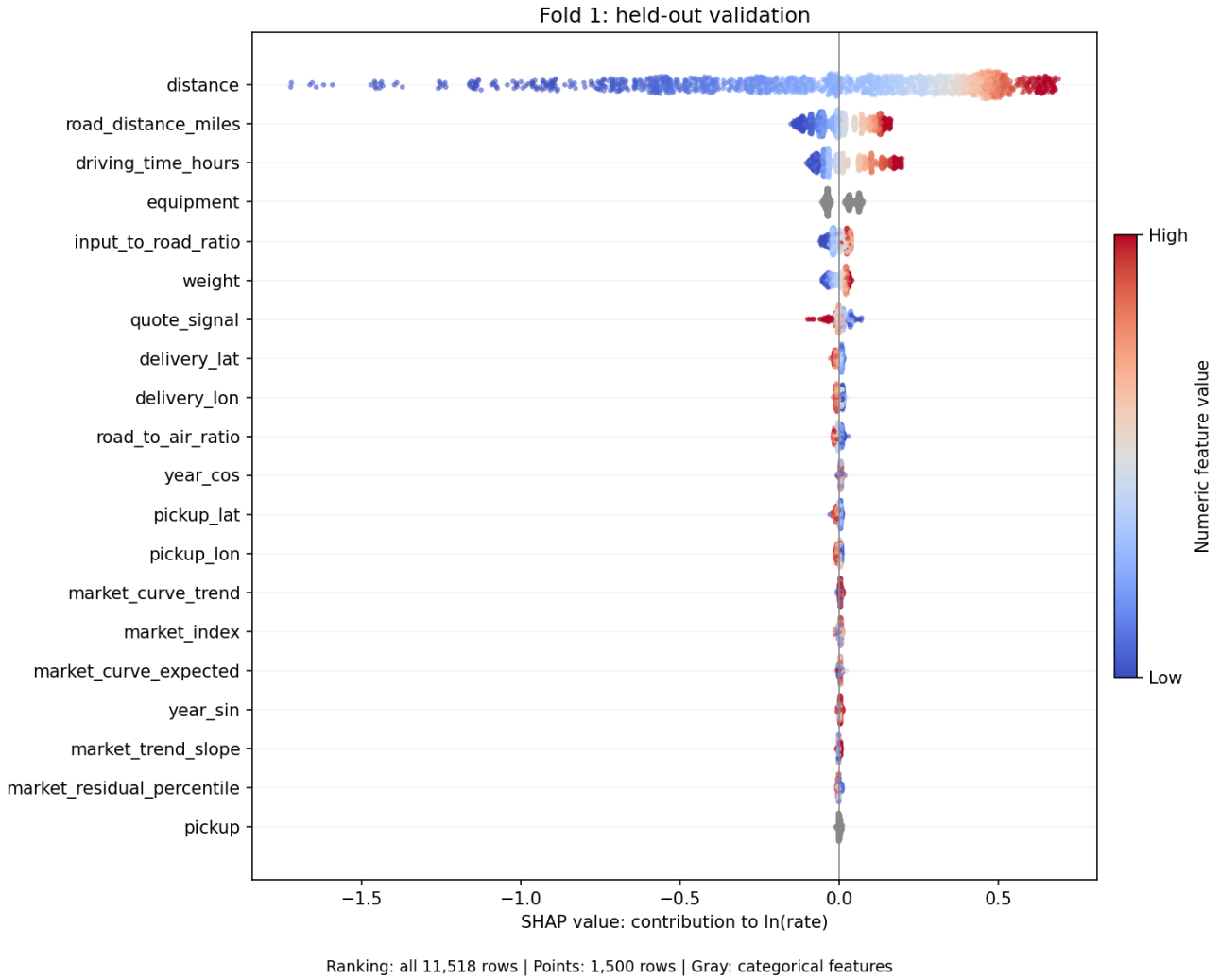


Figure 19: Fold 1 explanations on its validation rows, in log-rate units.

B Fold 2: held-out SHAP values

Training: January 1–May 26; validation: May 27–August 6. Fold MAPE is 5.006%, with 285 trees. Explanations use the fold model, not the submission model.

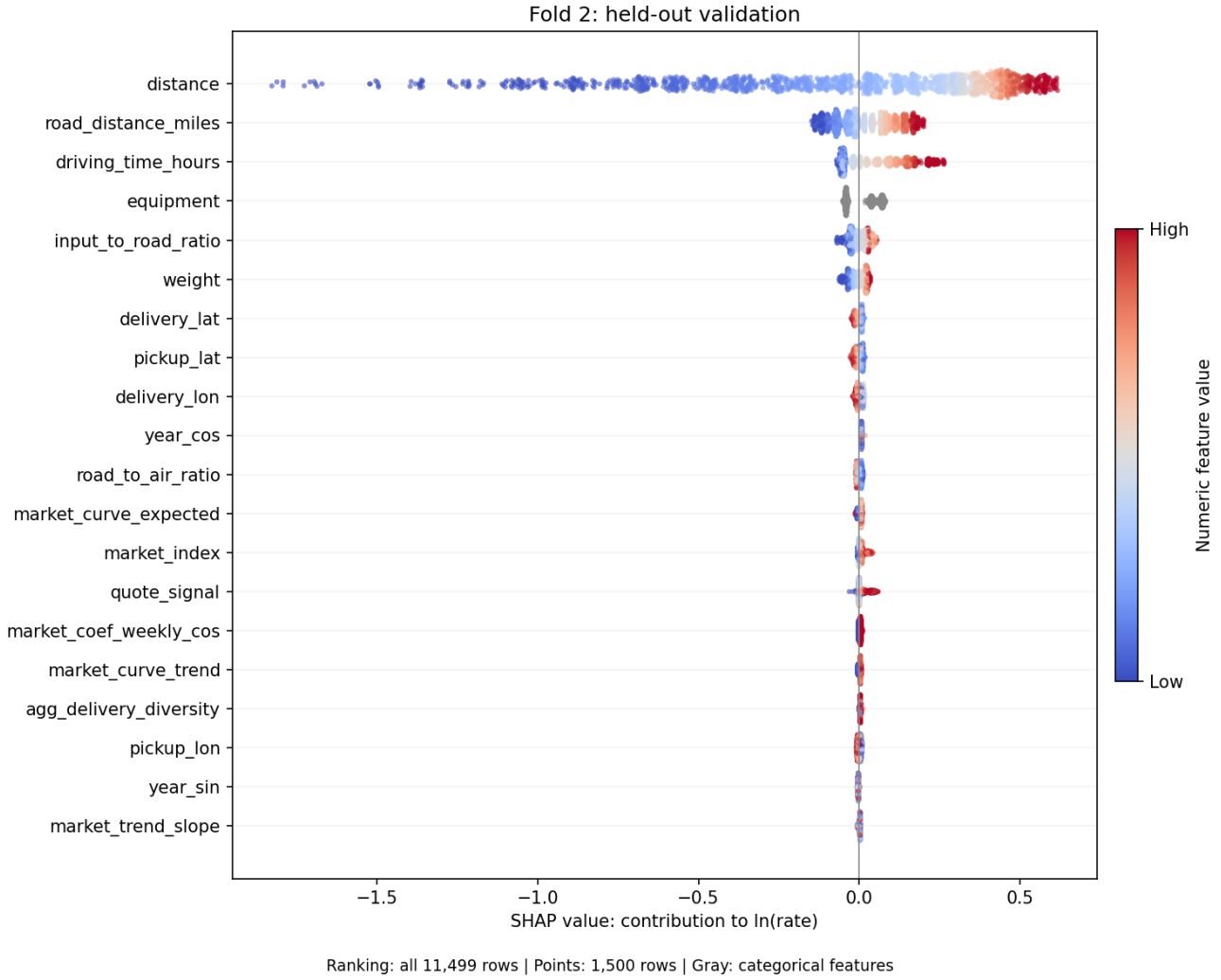


Figure 20: Fold 2 validation explanations; ranking uses all rows and point sampling is deterministic.

C Fold 3: held-out SHAP values

Training: January 1–August 6; validation: August 7–October 17. Fold MAPE is 4.740%, with 427 trees. The most recent fold receives weight 0.5 in the optimization objective.

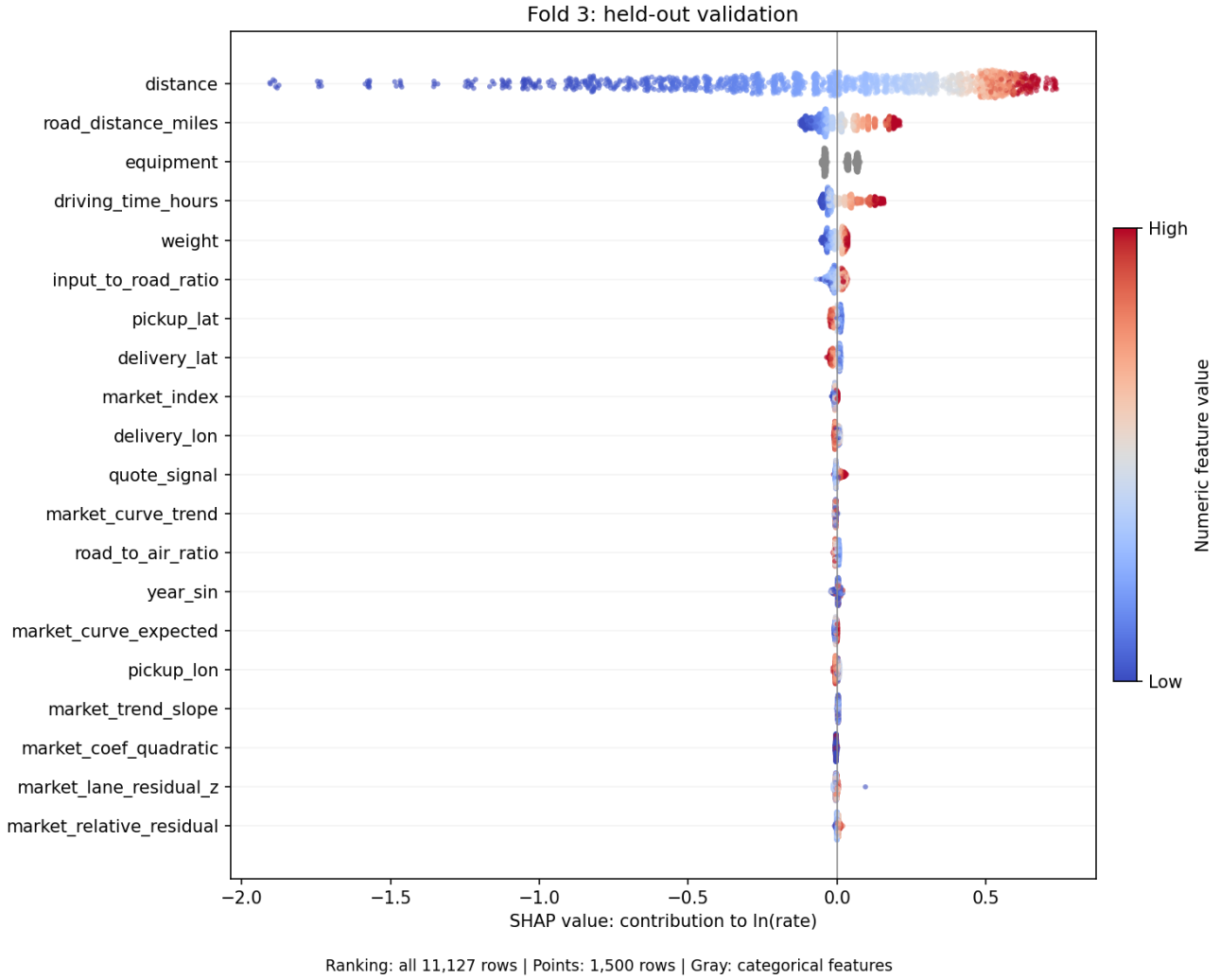


Figure 21: Fold 3 held-out explanations. Differences in attribution do not establish causal changes.

D Validation inference and sources

All 12,000 validation loads use the full-train model. Actual prices are unavailable. These graphs describe predictions only and do not measure accuracy.

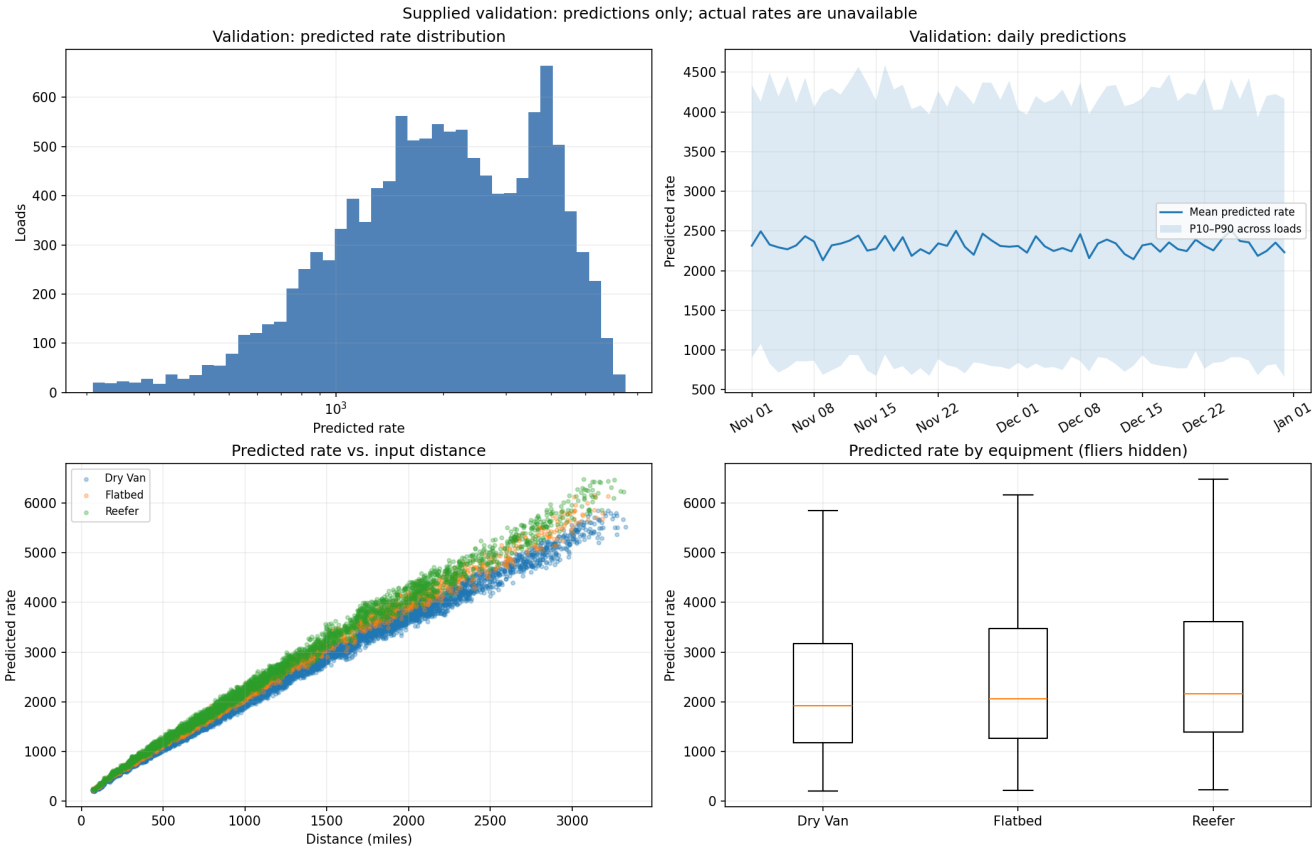


Figure 22: Validation predicted-price distribution, daily mean and P10–P90 across loads, distance dependence and equipment groups. Across-load spread is not an interval for one load.

Sources and evidence

- Assessment: local `data/README.md` and `notebooks/score.py`. The referenced assessment PDF is absent from the workspace.
- Results: executed notebooks, saved model metadata and current golden-feature results; `output/pdf/evidence.json` records evidence and hashes.
- City references: GeoNames cities15000, CC BY 4.0; records in `data/city-coordinates.csv`.
- Polygons: US Census TIGER/Line, 2025 assets; local build/audit logic is in `scripts/build_city_boundaries.py`.
- Routes: cached HERE truck estimates, snapshot `2026-10-12-city-routing-v2`. No credentials appear in report artifacts.
- Contribution semantics and additivity: CatBoost ShapValues documentation. Native values are plotted with Matplotlib.